

Large References, Small Checks: Amortized Testing of Stochastic Learning Systems

Anonymous authors

Paper under double-blind review

Abstract

Testing successive implementations of a stochastic learning system motivates an asymmetric allocation of effort: construct an expensive reference once, then reuse it for many smaller checks. We study an empirical outcome oracle that separates this upfront investment from recurring port-testing cost. Reference counts define an observed outcome set and a high-frequency head; short check samples report previously unobserved outcomes and selected reference outcomes that were not reproduced. Good–Turing estimates guide reference sampling heuristically, not as confidence certificates. We formalize amortized cost and distinguish independent single runs from fixed-horizon episodes that retain memory. A retrospective Metacat/Petacat case study covers both modes. Across 19 inputs, 374,500 single-run reference observations supported cycles of 1,900 initial checks. A reported reference of 9,500 eight-run learning-mode episodes supported cycles of 1,900 episodes. In saved CPU records, novel single-run pairs decreased from five to one after repairs, while episodes with a stored novel endpoint decreased from 28 to 14. No selected head member was missing, but episodic cap effects and incomplete historical reference provenance limit interpretation. These are asymmetric execution budgets, not measured wall-clock speedups. The contribution is a reusable diagnostic workflow for stochastic reimplementations, including memory-dependent behavior, and an account of its cost and statistical limits. Matched-cap episodic replication and independent calibration remain necessary before stronger reliability claims.

1 Introduction

Stochastic implementations are difficult to compare when their output distributions, rather than individual seeded trajectories, express the intended behavior. A language port may consume random draws in a different order, so matching seeds across implementations need not yield matching executions. Nevertheless, reimplementations can alter scientifically important outcomes. Researchers need ways to identify such changes without interpreting every disagreement between individual runs as a defect.

We study this problem through a Python reimplementations of Metacat (Marshall, 1999; Marshall & Hofstadter, 1997), an analogy-making architecture extending Copycat (Mitchell, 1993; Hofstadter & Fluid Analogies Research Group, 1995). For an input such as `abc -> abd; xyz -> ?`, probabilistically scheduled computational units, or *codelets*, construct and revise candidate relationships until the program produces an answer or reaches a stopping condition. Different answers can reflect different interpretations of the same analogy. This makes agreement with a particular answer an insufficient general test of the implementation.

The central engineering objective is **asymmetric cost over a development lifecycle**. A reference implementation may be stable while its port changes repeatedly. Paying for a large reference campaign at every revision would undermine the usefulness of regression checks. Instead, we invest in a versioned reference artifact once and reuse it for many smaller port samples. The question is not merely whether two samples differ, but what discrepancy information a short recurring check can extract from that prior investment.

Our question is operational: **can an expensive, reusable reference make repeated checks of a stochastic learning implementation small and informative?** We report two named-outcome flags. A NOVEL flag denotes a check outcome outside the *observed* reference set, not necessarily outside the reference’s

true support. A MISSING flag denotes a selected high-frequency reference outcome absent from the check, not proof that the port cannot reach it. Both require investigation.

Metacat’s episodic memory makes the learning mode essential to this question. Stored experiences affect later runs, so matching memory-free answers would leave an important part of the reimplementation untested. We therefore examine both memory-free runs and fixed-horizon episodes with memory retained between runs. Testing fidelity of that learning mechanism is distinct from showing that it improves task performance or transfers to unseen analogy problems.

The method combines differential testing, empirical head selection, and missing-mass estimation. These ingredients and asymmetric sample allocation have prior art; we claim neither a new estimator nor an optimal support-equality test. The contributions are:

1. A two-stage workflow that separates reusable reference construction from small recurring checks, with explicit amortized-cost accounting and conditions under which the reference remains reusable.
2. An episode-level formulation for testing memory-dependent learning behavior, alongside ordinary fresh-memory runs, using concrete outcome reports.
3. An analysis separating reference-distribution plug-in baselines from valid statements about support equality, including counterexamples to transferring those baselines under unrestricted probability reweighting.
4. A retrospective case study in both modes, including an arithmetic audit of archived counts and ordered port episodes, a documented defect investigation, and explicit limits on reproducibility.

The relevance to machine-learning research is implementation fidelity in stochastic scientific models. Analogy-making and probabilistic inference are settings in which a reproducible program matters to interpreting a scientific claim. The case study concerns a cognitive architecture’s stochastic reasoning and episodic learning behavior. It does not establish beneficial learning, convergence, or generalization to modern generative models. The wider lesson is how to evaluate recurring implementation checks when their reference is costly and the model’s state depends on experience. A distributional test remains necessary when the scientific claim concerns outcome probabilities.

2 Background and Related Work

The absence of an immediately available correctness oracle is an established software-testing problem (Weyuker, 1982; Barr et al., 2015; Patel & Hierons, 2018). A second implementation can serve as a pseudo-oracle (Davis & Weyuker, 1981) or differential-testing reference (McKeeman, 1998), but the interpretation of disagreement depends on the reference’s validity and on the semantics being compared. An intentionally stochastic model differs from a test whose nondeterminism is incidental flakiness (Luo et al., 2014).

Metamorphic testing replaces individual expected answers with relations between executions (Chen et al., 1998; 2018; Segura et al., 2016). Statistical metamorphic testing and work on stochastic optimization explicitly address random outputs (Guderlei & Mayer, 2007; Yoo, 2010). Such relations complement the present diagnostic when domain knowledge supports them. Statistical testing of randomized algorithms requires a specified null, effect size, and sampling protocol (Arcuri & Briand, 2014). Statistical model checking instead estimates or tests stated properties of stochastic executions (Younes & Simmons, 2002; Legay et al., 2010); its sequential guarantees do not follow merely from inspecting any statistic repeatedly.

Good–Turing estimation (Good, 1953) estimates probability mass assigned to outcomes not yet observed. Practical frequency estimation (Gale & Sampson, 1995), unseen-vocabulary estimation (Efron & Thisted, 1976), richness estimation (Chao, 1984), and coverage-based estimation (Chao & Lee, 1992) address related but distinct questions. Finite-sample missing-mass analysis supplies confidence-dependent error terms (McAllester & Schapire, 2000), not permission to treat an observed singleton ratio as a certified upper limit. The optimality results of Orlitsky et al. (2016) concern extrapolating unseen species counts, not the diagnostic or stopping rule studied here.

STADS (Böhme, 2018) and residual-risk estimation in greybox fuzzing (Böhme et al., 2021) connect species-discovery statistics with software testing. Capture-recapture methods have also been used for fault-content estimation (Eick et al., 1992). We apply a missing-mass estimate to a reusable reference outcome sample, while recognizing that guarantees developed for particular testing or adaptive-sampling schemes do not automatically apply to ours. In particular, STADS already treats program behaviors, including outputs, as species. The distinction evaluated here is the use of a frozen reference artifact for repeated, named-outcome port checks, not the invention of coverage-guided software testing.

Asymmetric sampling also has direct prior art. Bhattacharya & Valiant (2015) analyze distributional closeness testing with unequal sample sizes and heavy/light outcome separation. Their target is equality versus specified distributional separation, whereas our reports localize observed set differences without such a calibrated verdict. Frequency-sensitive methods can reuse reference samples too. The meaningful comparison is recurring cost and diagnostic behavior under stated targets, not reuse versus an alternative artificially forced to rebuild its reference. The count-ranked cumulative-mass head is an empirical smallest covering region (O’Neill, 2022), not a new selection algorithm; 50% is a design choice, not an established optimal testing threshold.

Alignment and replication of simulation models have long distinguished levels of agreement (Axtell et al., 1996; Wilensky & Rand, 2007). Observed reachability is weaker than distributional equivalence, and matching final answers is weaker than matching internal cognitive processes. Testing probabilistic programming systems (Dutta et al., 2018) is another relevant application area, but this paper does not evaluate the present diagnostic on a probabilistic programming benchmark. Reset-based finite-trace testing of probabilistic stateful systems is established (Gerhold & Stoelinga, 2018). It provides the appropriate comparison for our fixed-episode formulation. Missing-mass estimation for dependent Markov sequences (Pananjady et al., 2024) further shows why ordinary singleton counts cannot simply be pooled across an arbitrarily evolving learning history.

3 Asymmetric Cost and Reference Reuse

Let C_R denote the upfront cost of constructing, validating, and storing an empirical reference under a fixed protocol. Let $C_{P,k}$ be the cost of check cycle k , including sampling and comparison, and let $C_{F,k}$ be its follow-up investigation cost. Across K port-check cycles that can legitimately reuse the reference, total and average costs are

$$C_{\text{total}}(K) = C_R + \sum_{k=1}^K (C_{P,k} + C_{F,k}), \quad \bar{C}(K) = \frac{C_R}{K} + \frac{1}{K} \sum_{k=1}^K (C_{P,k} + C_{F,k}).$$

This accounting identity expresses the design objective: tolerate a substantial upfront investment to reduce recurring sampling and interpretation work. It is not itself evidence of a speedup. Reference validation, cap-resolution reruns, storage, and investigation must not disappear from the accounting merely because they occur outside the nominal short check. A baseline using the same reference investment must be allowed to amortize it on the same terms.

For a single-run protocol, the sampling asymmetry is N_x reference runs versus n_x runs per check, usually with $N_x \gg n_x$. For a fixed-horizon learning protocol, those units become whole episodes. An episode can be much more expensive than a memory-free run, especially when memory changes termination behavior. Run and episode counts describe allocation, while measured execution time or compute consumption is needed to describe actual cost. Neither unit automatically translates into a wall-clock advantage across Scheme, Python, CPU, GPU, or different concurrency settings.

The reusable artifact comprises counts, observed support, selected head, reference implementation and dependency identities, input schedules, initial memory, stopping conditions, seeds, and outcome definitions. Reuse is justified only while these define the same intended reference behavior. Changing an episode horizon, initial memory, cap, or interpretation of an outcome is not merely another port revision: it changes the ref-

reference question and requires separate construction or a justified transformation of the artifact. A confirmed reference defect also invalidates affected portions of the reference.

Reference reuse across adaptively developed revisions does not produce independent evidence or a family-wide false-alarm guarantee. Developers may overfit a fixed diagnostic. Independent final audit samples and known-defect controls are therefore important even when the routine oracle remains fixed. The intended contribution is an economical investigation workflow, not a claim that unlimited reuse preserves statistical error control.

4 Diagnostic Construction

4.1 Outcomes, Samples, and Scope

For input x , let $p_R(o \mid x)$ and $p_P(o \mid x)$ denote the reference and port probabilities of outcome o in a countable outcome space $\mathcal{O}$. The true support is $\text{supp}(p) = \{o : p(o) > 0\}$. A reference sample of N_x runs supplies counts $c_x(o)$ and the observed set

$$S_x = \{o : c_x(o) > 0\}, \quad \hat{p}_R(o \mid x) = c_x(o)/N_x.$$

An outcome observed under the specified protocol is evidence of reachability; an unobserved outcome need not be impossible. All probabilities are conditional on a fixed implementation, configuration, input, stopping protocol, and outcome definition. We use n_x for the check size and J for the number of inputs, avoiding reuse of the reference sample size or program symbol.

The single-run projection records an answer string, or ***NONE*** for stopping without an answer, or ***CAP*** for exhausting the codelet budget. These reserved outcomes remain in the counts and head selection. The projection discards internal structures, their construction order, and timing. It therefore cannot establish process-level fidelity. A larger or infinite outcome space need not prevent a small missing mass, but a highly diffuse distribution may make this projection unhelpful at an affordable budget.

4.2 Learning Episodes as Observations

For episodic learning, the reference condition includes an initial memory M_0 , an input schedule, and a fixed horizon L . Memory persists between runs within an episode and is reset to M_0 before the next episode. Write $\mathcal{E}_{x,e}$ for episode e 's ordered outcomes and memory evolution, and choose a categorical projection $Z_{x,e} = \phi(\mathcal{E}_{x,e})$ before sampling. An episode endpoint, a complete answer sequence, or a declared summary of memory-dependent events defines a different projection and therefore a different oracle.

The same set construction applies to the projected observations:

$$c_x^\phi(z) = \sum_{e=1}^{N_x} \mathbf{1}\{Z_{x,e} = z\}, \quad S_x^\phi = \{z : c_x^\phi(z) > 0\}.$$

Here N_x counts whole episodes, not the LN_x dependent runs inside them. Fixed-protocol, independently randomized reset episodes can sample a common episode law, subject to checking for hidden cross-episode state. This is the reset-trace perspective of Gerhold & Stoelinga (2018), not a new independence theorem. Singleton counts and head selection are calculated on projected episodes. The guarantees or caveats for that law do not apply to a pool of all within-episode steps.

The historical learning-mode projection is the last successful answer within eight runs. We call this a *finite-horizon endpoint*, not convergence. A total projection must explicitly retain the case of an episode with no answer. Adding final-run status also distinguishes an answered episode whose last run caps from one that ends with an answer. The archived endpoint comparison and its exclusions are reported in Section 6.5; they are not silently replaced with a newly defined historical reference.

A terminal projection can hide changes in learning dynamics. Matching it does not establish matching retrieval, memory updates, or stepwise behavior. Full trajectories and memory-specific controls are necessary to test those claims. Continually growing memory without between-episode resets poses a different dependent, potentially nonstationary problem; stationary Markov missing-mass results (Pananjady et al., 2024) do not automatically resolve it.

4.3 Reference Sampling Is Heuristic

Let $f_{1,x} = |\{o : c_x(o) = 1\}|$. The actual reference missing mass and its Good–Turing estimate are different objects:

$$M_{R,x} = \sum_{o \notin S_x} p_R(o | x), \quad \widehat{M}_{R,x} = f_{1,x}/N_x.$$

The estimate is informative about the reference’s unobserved probability mass, not about the fraction of all possible outcomes that has been discovered. It is not an upper confidence bound. In particular, the finite-sample analysis of McAllester & Schapire (2000) concerns independent sampling with confidence-dependent error terms; it does not certify optional stopping on the raw estimate.

The archived single-run sampler inspected merged batch counts and used the following operational rules: after at least 3,000 runs, stop if $f_{1,x} > 0$ and $f_{1,x}/N_x \leq 10^{-4}$; if $f_{1,x} = 0$, require at least 10,000 runs before stopping. A lower reporting threshold of 6×10^{-5} recorded overshooting, not a second acceptance condition. A 50,000-run checkpoint and a 200,000-run hard maximum limited expenditure. Eight workers sampled disjoint interleaved seed indices, while a coordinator merged counts. In-flight work could raise the final count beyond a threshold. Global singleton counts, not independent worker decisions, governed the stopping statistic.

These rules choose a practical reference size. The 10,000-run floor is a guard against stopping very early, not a theorem. For example, with two outcomes having probabilities 0.9998 and 0.0002, all 10,000 draws equal the first outcome with probability

$$(0.9998)^{10000} \simeq 0.1353.$$

Then $f_1 = 0$ and the floor is satisfied, yet the missing mass is 2×10^{-4} , twice the target. Also, $f_1 = 1$ at $N = 10,000$ satisfies the positive-singleton rule; the floor is not justified by rejecting that case.

No empirical superiority over fixed-budget or no-discovery stopping is claimed. The available reference record contains outcome counts and codelet histograms, not the ordered discoveries needed for that comparison. An earlier development account of 35 outcomes and six long discovery gaps could not be reconciled with the archived 10-outcome sample for the same named input and is excluded from the evidence here.

4.4 Head Selection and Flags

Sort observed outcomes by decreasing count, breaking ties lexicographically by outcome key. Let H_x be the shortest prefix whose cumulative count is at least $N_x/2$. We call this the empirical *p50 head*. This defines a coverage-based selection rule, not a lower bound on every selected outcome’s probability. For a uniform distribution over B outcomes, a member’s probability is $1/B$; even a head member can be arbitrarily rare as B increases. Selection is also uncertain: the reference outcome `copy5/cc` lies exactly at the empirical half-mass boundary, so resampling could change the head size.

Freeze S_x and H_x before checking the port. For the check’s observed set T_x , report

$$\text{MISSING}_x = H_x \setminus T_x, \quad \text{NOVEL}_x = T_x \setminus S_x.$$

MISSING prioritizes a selected reference outcome that was not reproduced; NOVEL prioritizes an outcome not previously observed in the reference. Neither is a verdict of correctness. No flag means only that all selected head outcomes were seen and all check outcomes belonged to the observed reference set.

The check compares membership, but frequency determines head selection and the optional baseline calculations. A flag report should retain its input, outcome, count, seeds, configuration, reference sample size, stopping reason, and missing-mass estimate. Novel outcomes are not automatically added to the reference: follow-up may reveal an incomplete reference, a reference defect, a port defect, or a protocol mismatch.

For each input x :

- Collect reference counts under the recorded budget/stopping rule.
- Freeze S_x = observed reference keys.
- Freeze H_x = the count-ranked prefix covering at least half the sample.
- Record counts, singleton ratio, stopping reason, and configuration.

For each check cycle and input x :

- Collect n_x independent-unit port outcomes under a compatible protocol.
- The unit is a fresh-memory run or a whole reset learning episode.
- Report H_x minus observed keys as MISSING.
- Report observed keys minus S_x as NOVEL.
- Attach evidence and optional reference-distribution plug-in baselines.
- Investigate flags; do not infer support equality from a quiet cycle.

5 Statistical Interpretation

5.1 Absence Under the Check Distribution

Conditional on the frozen reference sample, and assuming independent check observations from a stationary distribution, the absence probability for $o \in H_x$ is

$$\Pr(o \notin T_x \mid S_x, H_x) = (1 - p_P(o \mid x))^{n_x}.$$

Substitution of $\hat{p}_R(o \mid x)$ gives a *plug-in reference-distribution baseline*, not a calibrated probability under the null of equal supports. Even under equality of distributions, the plug-in quantity does not account for uncertainty in the reference shares, selection of the head, or reference stopping. Absence of a still-reachable outcome is a false-positive discrepancy flag under a matched-distribution null; it is not the false-negative probability for detecting deletion of that outcome. If $p_P(o \mid x) = 0$, it is absent with certainty in any nonempty check sample.

Counterexample under equal supports. A head outcome with reference share approximately 0.1947 may have port probability 0.001 while both true supports remain identical. Its absence probability in 100 port runs is $0.999^{100} \simeq 0.9048$, not approximately 4×10^{-10} .

If one could justify a lower bound $p_P(o \mid x) \geq \ell_{x,o} > 0$, then $(1 - \ell_{x,o})^{n_x}$ would be an absence upper bound. A union bound over all selected members gives a cycle bound by summing those quantities, without requiring independence between member-absence events. Justifying the lower bounds, including any estimation uncertainty, is separate work. Support equality alone supplies no such positive quantitative lower bound.

5.2 Novelty and Its Counting Units

For a frozen S_x , define $q_{P,x} = \sum_{o \notin S_x} p_P(o \mid x)$. For independent check draws, the expected number D_x of out-of-set draws and the probability of flagging the input are

$$\mathbb{E}[D_x \mid S_x] = n_x q_{P,x}, \quad \Pr(D_x > 0 \mid S_x) = 1 - (1 - q_{P,x})^{n_x}.$$

The harness instead lists *distinct* novel problem–outcome pairs. If U_x is their count, then

$$\mathbb{E}[U_x \mid S_x] = \sum_{o \notin S_x} [1 - (1 - p_P(o \mid x))^{n_x}] \leq n_x q_{P,x}.$$

These are different counting units. Under $p_P = p_R$, $q_{P,x} = M_{R,x}$, which can be estimated by the reference singleton ratio. If only the supports match, however, arbitrarily much port mass can be assigned to reference outcomes not yet observed. The reference missing-mass estimate then need not approximate the check’s novelty probability. Likewise, even an exactly known reference missing mass would not transfer without an assumption controlling probability drift. For example, a justified total-variation bound of d_x would imply $q_{P,x} \leq M_{R,x} + d_x$; no such bound has been established for this port.

If erroneous outcomes have total port mass ϵ and all lie outside S_x , the probability of missing all of them in n_x independent runs is $(1 - \epsilon)^{n_x}$. At $n_x = 100$, detection probabilities for $\epsilon = 10^{-4}, 10^{-3}, 10^{-2}$ are approximately 0.0100, 0.0952, and 0.6340. Short checks can therefore overlook rare defects even when observed head outcomes are easy to reproduce.

5.3 A Frequency-Sensitive Comparator

Membership diagnostics are not substitutes for frequency-sensitive tests, nor are such tests inherently expensive or uninterpretable. Consider a known Bernoulli reference law with $p_R(A) = 0.85$ and a port with $p_P(A) = 0.20$; the complementary outcome has positive probability in both. The reference p50 head is $\{A\}$. A 100-run membership check misses A with probability only $0.8^{100} \simeq 2.04 \times 10^{-10}$, despite the substantial rate change.

For comparison, the explicitly specified binomial test rejecting the known null $p(A) = 0.85$ when $C_A \leq 50$ in 100 draws has null rejection probability

$$\sum_{k=0}^{50} \binom{100}{k} (0.85)^k (0.15)^{100-k} \simeq 2.30 \times 10^{-16},$$

and power approximately 0.999999999995 at $p(A) = 0.20$. These are exact toy-distribution calculations, not a comparison on the Metacat benchmark. Estimating an unknown reference law would require accounting for that estimate. The example establishes the narrower point: low check cost is not unique to membership diagnostics, and a change the latter usually ignores may be easy for a frequency-sensitive test to detect.

6 Retrospective Case Study

6.1 Systems and Protocol

The Python port was developed through model-assisted translation and repeated comparison with the Scheme implementation and dissertation. Runtime checking complemented source inspection; neither was sufficient alone. The benchmark contains 19 letter-string inputs drawn from demonstrations and simpler analogy cases. Appendix C lists all inputs and selected shared outcomes. Single-run checks start with cleared episodic memory. Learning-mode checks instead retain memory through eight-run episodes and clear it between episodes (Section 6.5). Independence and stationarity of pseudorandomly indexed sampling units remain modeling assumptions, not consequences of clearing memory alone.

The historical reference sampler used a locally modified, headless Metacat 1.2 with a 100,000-codelet cap. The source changes address unsupported object messages, whole-string rule handling, dispatch, and display-free execution. Some changes can affect outcomes; the reference is not represented as unmodified Metacat or as distributionally equivalent to it.

A patch-only reconstruction bundle now makes the modified source available without redistributing the original tarball. However, that current source snapshot does not prove which exact build generated the

archived reference sample. The records lack sufficient historical build attribution for an independent end-to-end replication. We report their counts as archived measurements and distinguish the new arithmetic audit from new engine runs.

The saved single-run checks use seeds 900,000–900,099 for each input. Reusing this block before and after repairs creates a paired replay comparison, not independent cycles. Different random-number generators also mean that equal numerical seeds would not pair trajectories between Scheme and Python.

6.2 Reference Summary

The archived reference contains 374,500 runs, 366 observed problem–outcome pairs, and 27 selected head members. A repeated answer on different inputs is counted separately. Reference sizes range from 10,250 to 51,000, averaging 19,710.5 per input. Six inputs have no singletons; three have estimated missing mass above 10^{-4} . Table 1 reports the per-input figures.

Table 1: Archived reference counts. Stopping labels describe the heuristic, not certified completeness.

Input	N_x	$ S_x $	$f_{1,x}$	$f_{1,x}/N_x$	Stored reason	$ H_x $
misc4	11,000	4	0	0.00000000	no_singletons	2
fig5.7	18,125	18	1	0.00005517	saturated	2
misc3	19,000	54	4	0.00021053	shards_exited	2
misc5	11,125	8	1	0.00008989	saturated	3
misc2	11,000	6	1	0.00009091	saturated	1
run1	11,000	11	0	0.00000000	no_singletons	1
run4	11,000	10	1	0.00009091	saturated	2
misc1	30,750	18	3	0.00009756	saturated	1
fig5.4-top	51,000	61	19	0.00037255	checkpoint	2
eqe-baaab	51,000	51	13	0.00025490	checkpoint	1
run6	47,500	80	4	0.00008421	saturated	1
run3	21,000	12	2	0.00009524	saturated	2
run2	11,000	10	1	0.00009091	saturated	1
copy1	10,875	2	0	0.00000000	no_singletons	1
copy2	11,250	1	0	0.00000000	no_singletons	1
copy3	11,000	1	0	0.00000000	no_singletons	1
copy4	11,125	2	0	0.00000000	no_singletons	1
copy5	15,500	13	1	0.00006452	saturated	1
copy6	10,250	4	1	0.00009756	saturated	1

The stored **saturated** label means the *heuristic* threshold was met, not that the true support was enumerated or the true missing mass bounded. The two checkpoint cases end at 51,000 after in-flight work. The remaining above-target case, **misc3**, has stored reason **shards_exited**; development notes describe a manual stop, but the aggregate file does not by itself establish why workers exited. Heterogeneous sample sizes motivate adaptive allocation, but do not demonstrate that this allocation outperforms a fixed-budget alternative.

For $n_x = 100$, the smallest selected empirical share is $4089/21000 = 0.194714\dots$ for **run3/wyz**. Its plug-in absence probability is 3.9355×10^{-10} . Summing the 27 member-specific plug-in absence probabilities yields 6.1340×10^{-10} . This is the cycle union-bound calculation *after substitution of empirical reference rates*, not a confidence guarantee for the real port.

Using the actual singleton ratios produces three distinct plug-in summaries: 0.169485 expected novel draws per cycle, 0.167898 expected inputs with at least one novelty, and probability 0.155914 of any novelty if check samples are also independent across inputs. The last calculation is $1 - \prod_x (1 - \bar{M}_{R,x})^{100}$. The expected distinct-pair count is bounded by the expected draw count under the corresponding plug-in model, not generally equal to it. Zero-singleton inputs contribute zero to these plug-in sums, not certified zero risk.

The design quantity $19 \times 100 \times 10^{-4} = 0.19$ is not an established bound: three recorded estimates exceed the target, and none is an upper confidence limit. The actual plug-in sum happens to be below 0.19.

6.3 Cap Compatibility and Cost

Each check cycle schedules 100 initial runs per input, or 1,900 in total. The reference-to-initial-check run-count ratio is $374500/1900 = 197.105 \dots$. It is not a measured wall-clock speedup, and the one-time reference cost must be paid again if its defining implementation or protocol changes.

The port first runs at a 20,000-codelet working cap. In the repaired single-run harness, capped seeds are rerun at the reference’s 100,000-codelet cap. The saved post-repair cycle records 23 such reruns, so it entails 1,923 execution attempts, not exactly 1,900. The reported 197-fold ratio excludes these extra attempts and the different work per attempt. Parallel worker counts likewise do not establish proportional wall-clock speedups.

Cap reconciliation is valid as trajectory extension only if seeded replay is deterministic and the larger cap does not alter transitions before the original cutoff. Fresh sessions alone do not guarantee this. Differences in codelet semantics or numerical backends can remain even with equal numeric caps. The case study does not establish general CPU/GPU equivalence.

6.4 Flags and Investigation

Table 2 summarizes the three archived single-run checks. Each contained every selected reference head member. This establishes observed reachability in those check samples, not comparable outcome rates, support equality, or fidelity over all development cycles.

Table 2: Observed flags, not estimated error rates. All cycles reuse the same seed block.

Archived cycle	Novel pairs	Missing head members
CPU before repairs	5	0
CPU after repairs	1	0
Archived MLX check	5	0

The pre-repair CPU cycle reported five novel pairs: `ege-baaab/abbbb`, `run6/cdddb`, `copy1/*NONE*`, `run1/*CAP*`, and `copy5/aac`. Development traces associated the first three with one direction-sensitive image defect (RC-A). A leftward group’s image was constructed in physical left-to-right order with rightward direction. Rendering could conceal the mismatch because reversing both direction and constituent order leaves an untouched string unchanged; subsequent direction-sensitive rule operations exposed it.

The development record contains a Python trace and Scheme-side probes. Injecting the image defect into the reference reportedly produced `cdddb` and `*NONE*`, matching two of the flagged outcomes. The quoted intervention table does not show reproduction of `abbbb`; that finding is supported by the Python trace and its disappearance after repair, not an assertion that every flagged string was reproduced in Scheme. Directional image regression tests provide additional local evidence. The historical instrumented runs have not been independently repeated in this revision.

The `run1/*CAP*` finding (RC-B) arose from comparing different resource caps and disappeared after cap reconciliation. Following the image and harness repairs, the archived CPU replay contained only `copy5/aac`, whose proposed cause was investigated and rejected (RC-C). It remains unresolved. The archived MLX cycle still contains five novel pairs and does not demonstrate that those repairs were validated on that backend.

A cross-cycle recurrence marker helps prioritize investigation, but supplies no proof of invalidity. A legitimate reference outcome omitted from S_x can recur indefinitely. In these saved cycles, reuse of the same seed block is an additional reason not to interpret recurrence as independent confirmation.

6.5 Episodic Learning-Mode Checks

Learning-mode comparison applies the same asymmetric design to repeated memory-bearing sessions. The historical report describes a reference of 500 episodes per input, or 9,500 episodes across the 19 inputs, each consisting of eight runs on the same problem. The port checks use 100 episodes per input: 1,900 episodes and 15,200 within-episode runs per cycle. One memory is retained inside each episode and a fresh memory is used for the next. The reference therefore represents 76,000 within-episode runs and a 5:1 reference-to-check episode-count ratio. This is a smaller asymmetry than the approximately 197:1 single-run ratio; the latter must not be presented as a learning-mode result. Both reference artifacts are intended for reuse across check cycles.

The 500-episode reference budget should not be confused with satisfying the single-run sampling heuristic in Section 4.3. Stored episodic singleton ratios range up to 0.016, and the episode reference has no independent missing-mass validation in the available record.

Memory mechanism. The port retains answer and snag descriptions across runs while reinitializing the workspace, concept network, scheduler, and per-run reminding activations. Its direct cross-run search feedback is duplicate-answer rejection: the problem strings, answer string, and structural signatures of both the inferred rule and its target translation determine whether an answer has already been stored. The answer finder, justification, and justification-clamp recovery paths consult this memory. A matching answer is rejected, allowing search to continue; the same answer letters obtained by a different rule are not necessarily duplicates. Retaining memory can therefore change both later outcomes and whether a run terminates (Marshall, 1999; Marshall & Hofstadter, 1997). An endpoint over answer letters necessarily loses some of the information that drives this mechanism, so even matching complete letter sequences would not alone verify rule-level memory fidelity.

Reminding and comparison constitute a second, distinct behavior. After an answer is stored, the inspected reporting path compares it with past answers using themes, rules, justification, and coherence, then records graded activations and emits reminding commentary. Stored snag descriptions can also change the retrospective explanation of an otherwise unjustified theme. These explanatory functions are part of the implemented architecture, but are not evidence that retrieval improves subsequent answer quality. Here, learning-mode testing concerns experience-dependent behavior, principally the cross-run duplicate guard, rather than parameter fitting or a demonstrated learning curve. The repeated-problem protocol measures memory-conditioned exploration; it does not by itself measure transfer to new problems or an externally assessed gain in answer quality.

Implementation-level checks. Existing regression definitions address several memory-specific repairs: distinguishing answers reached by different rules; preventing justification-clamp recovery from accepting a remembered answer; comparing snag rules structurally instead of by potentially colliding English descriptions; and giving a past answer zero reminding activation at the distance threshold. Other checks cover the distance components, snag-based reclassification of themes, clearing activations without deleting stored answers, and session-scoped identifiers. The episode harness also requires both an answer-found status and a newly added memory entry before counting a run as successful; an answer left in memory by an earlier run is insufficient. These inspected checks document intended local contracts, not a freshly executed regression result or Scheme/Python trajectory comparison. The archived endpoint changes below cannot be attributed to each of these repairs: their historical build attribution is incomplete.

The saved check files contain all ordered eight-state sequences. We recompute the last successful answer by searching backward past `*NONE*` and `*CAP*`, then verify the saved endpoint histogram and missing-head list. Four episodes per cycle never answer; the legacy histogram omits them but records their count separately. We retain that count in Table 3, rather than representing the success-only histogram as a complete episode distribution. The historical episodic reference itself is not present in the arithmetic supplement, so its 9,500-episode size is attributed to the development record, and stored novelty lists cannot be independently reconstructed here. The count of each listed novel endpoint is verifiable from the port trajectories.

Table 3: Archived eight-run episodes. Novelty labels come from stored flags; the historical episodic reference is unavailable for independent re-computation.

Metric	CPU before	CPU after	Archived MLX
Check episodes	1,900	1,900	1,900
Within-episode runs	15,200	15,200	15,200
Stored novel endpoint pairs	17	13	17
Episodes with stored novel endpoint	28	14	28
Missing stored head members	0	0	0
Episodes never answering	4	4	4
Capped within-episode runs	1,311	2,404	1,309
Episodes with any capped run	628	1,052	626
Episodes ending in a capped run	361	591	361

The paired CPU records contain 17 stored novel input/endpoint pairs across 28 episodes before repair and 13 pairs across 14 episodes afterward. No selected head member is missing in either cycle. Among the post-repair pairs, 11 also appear in the separate single-run reference; the remaining two are `misc2/ajd` and `eqe-baaab/qrrbq`, each in one episode. This cross-mode classification is useful context, not evidence that an outcome is correct for a particular memory history. An outcome reachable without memory need not have the same probability, or even be reachable under a chosen learned context.

The historical investigation connects the direction-image repair to the disappearance of `abbbb`, `baaba`, `cdddb`, and `cddbc` from the episodic novelty list. That record complements the single-run findings, but the exact instrumented historical builds and interventions have not been independently replayed. The archived MLX sequence data are a separate recorded cycle, not validation of a repaired GPU learning implementation.

Two limitations prevent interpreting this as a matched-protocol learning validation. First, the port uses 20,000 codelets per run while the reported reference uses 100,000. A cap early in an episode can alter memory and all subsequent runs. Similar aggregate cap frequencies would not establish that the mismatch is harmless. Moreover, CPU capped runs increase from 1,311 to 2,404 in the saved before/after records, even while novel endpoints decrease. The report’s later prose gives 2,406 capped runs and 1,054 affected episodes; the saved post-repair sequences give 2,404 and 1,052. We use the identifiable saved artifact rather than infer an undocumented intermediate cycle.

Second, episode e , step i uses seed $900000 + 8e + i$, with $0 \leq e < 100$ and $0 \leq i < 8$. Before/after cycles are paired, and these seeds overlap the single-run check block. They are not independent replications across modes. The episode endpoint is a useful historical diagnostic, but neither its agreement nor a lower flag count establishes preserved learning dynamics.

The next learning-mode evaluation must therefore use independently versioned, matched-cap episode references, held-out reference checks, and port checks, with all failures included in declared projections. Memory-retained and memory-cleared controls, and known memory-specific defects, test whether a small recurring check actually detects changes to the learning mechanism. Its construction, calibration, and recurring costs must be reported in whole episodes and measured compute, rather than borrowing the memory-free cost ratio.

7 Limits and Further Evaluation

Calibration. This is a diagnostic case study, not a calibrated test of unrestricted support equality. Reference-distribution plug-in quantities omit reference estimation, head-selection, and stopping uncertainty. Missing rare defects is possible; observing every head member does not bound drift in their probabilities. No frequency-based null test has been run on independent Metacat check blocks in this revision.

Historical evidence. Available single-run aggregates and ordered port episodes support the arithmetic and saved flag comparisons. They do not recover reference discovery order or the historical episodic reference,

identify all sampled builds, or independently reproduce the causal interventions. Claims of end-to-end experimental reproducibility require those artifacts or a new, prospectively versioned experiment. Providing current source diffs does not retroactively establish sample provenance.

Selection and scope. Inputs and reference/check parameters were selected during development, not pre-registered. The benchmark is small and belongs to one architecture. The learning-mode endpoint audit does not establish faithful memory trajectories, beneficial learning, or transfer to new problems. Continuous outputs, high-dimensional outputs, and adaptive generative pipelines are not evaluated. Scheduler, precision, or concurrency changes cannot be assumed to preserve support merely because they were intended as legitimate engineering changes. Outcome-level agreement can also conceal different internal processes.

Comparative claims. Fixed-budget sampling, no-discovery stopping, and frequency-sensitive comparisons deserve empirical evaluation under specified cost and power targets. The toy binomial comparison in Section 5.3 illustrates a distinction; it is not such a benchmark. The cost identity in Section 3 specifies the intended amortization, but the historical record does not measure all construction, validation, checking, and follow-up costs across revisions. Run-count ratios do not establish a speed advantage over other reusable-reference tests.

A prospective evaluation would pin both implementations and configurations, retain ordered per-run outcomes and seeds, construct the reference on one sample, and evaluate a frozen reference on fresh unchanged-reference and port seed blocks. Known defect injections would assess detection power, and equal-budget reusable-reference baselines would test whether the allocation rule adds value. The learning-mode study also requires matched caps, independent reset episodes, and memory-on/off and known memory-defect controls. Fresh check blocks are also needed before giving recurrence a probabilistic interpretation.

An independent fixed-size validation stage offers one way to separate reference construction from missing-mass inference. With a frozen S_x and m independent validation draws from the same reference distribution, zero out-of-set draws give a one-sided $(1-\alpha)$ binomial upper confidence limit $1-\alpha^{1/m}$ for $M_{R,x}$. At $\alpha = 0.05$, making that limit at most 10^{-4} requires at least 29,956 draws *per input*, with no repeated-look selection of m . Simultaneous coverage over inputs requires an additional confidence allocation. For eight-run learning episodes, 29,956 validation episodes per input across 19 inputs would require 4,553,312 runs, before reference construction or port checking. This cost is part of the upfront investment, not free calibration. A versioned single-run study is in progress; no results from it are included here, and it does not replace a learning-mode replication. The archived evidence contains no independent validation stage, and even that bound would not transfer to an arbitrarily reweighted port.

8 Conclusion

The intended economy is asymmetric: invest in an expensive empirical oracle once, then reuse it across smaller checks as a stochastic implementation evolves. An observed reference set and an empirical head make those checks concrete and inexpensive to evaluate, while Good–Turing estimates provide a heuristic for directing reference sampling rather than certifying completeness. The archived Metacat/Petacat evidence illustrates this workflow for both fresh-memory runs and fixed-horizon episodes with memory retained. The latter tests behavior of the learning mode, an essential part of reimplementing this architecture, rather than merely its memory-free solver.

The diagnostic directed attention to a direction-sensitive image defect and retained unresolved discrepancies. The episode audit also exposes the danger of treating last successful answers as convergence or overlooking cap changes. The present contribution is a cost-explicit testing workflow and a qualified case study, not a claim of a new missing-mass estimator, improved learning, or equivalence. Matched-cap episodic replication, independent calibration, and cost/power comparisons against other reusable-reference methods are needed to establish the wider value of this amortization strategy.

References

- Andrea Arcuri and Lionel Briand. A hitchhiker’s guide to statistical tests for assessing randomized algorithms in software engineering. *Software Testing, Verification and Reliability*, 24(3):219–250, 2014. doi: 10.1002/stvr.1486. URL <https://doi.org/10.1002/stvr.1486>.
- Robert Axtell, Robert Axelrod, Joshua M. Epstein, and Michael D. Cohen. Aligning simulation models: A case study and results. *Computational and Mathematical Organization Theory*, 1(2):123–141, 1996. doi: 10.1007/BF01299065. URL <https://doi.org/10.1007/BF01299065>.
- Earl T. Barr, Mark Harman, Phil McMinn, Muzammil Shahbaz, and Shin Yoo. The oracle problem in software testing: A survey. *IEEE Transactions on Software Engineering*, 41(5):507–525, 2015. doi: 10.1109/TSE.2014.2372785. URL <https://doi.org/10.1109/TSE.2014.2372785>.
- Bhaswar Bhattacharya and Gregory Valiant. Testing closeness with unequal sized samples. In *Advances in Neural Information Processing Systems*, volume 28, 2015. URL <https://proceedings.neurips.cc/paper/2015/hash/5cce8dede893813f879b873962fb669f-Abstract.html>.
- Marcel Böhme. STADS: Software testing as species discovery. *ACM Transactions on Software Engineering and Methodology*, 27(2):7:1–7:52, 2018. doi: 10.1145/3210309. URL <https://doi.org/10.1145/3210309>.
- Marcel Böhme, Danushka Liyanage, and Valentin Wüstholtz. Estimating residual risk in greybox fuzzing. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, pp. 230–241, 2021. doi: 10.1145/3468264.3468570. URL <https://doi.org/10.1145/3468264.3468570>.
- Anne Chao. Nonparametric estimation of the number of classes in a population. *Scandinavian Journal of Statistics*, 11(4):265–270, 1984. URL <https://www.jstor.org/stable/4615964>.
- Anne Chao and Shen-Ming Lee. Estimating the number of classes via sample coverage. *Journal of the American Statistical Association*, 87(417):210–217, 1992. doi: 10.1080/01621459.1992.10475194. URL <https://doi.org/10.1080/01621459.1992.10475194>.
- Tsong Yueh Chen, S. C. Cheung, and S. M. Yiu. Metamorphic testing: A new approach for generating next test cases. Technical Report HKUST-CS98-01, Department of Computer Science, Hong Kong University of Science and Technology, 1998. URL <https://arxiv.org/abs/2002.12543>. Reissued on arXiv in 2020.
- Tsong Yueh Chen, Fei-Ching Kuo, Huai Liu, Pak-Lok Poon, Dave Towey, T. H. Tse, and Zhi Quan Zhou. Metamorphic testing: A review of challenges and opportunities. *ACM Computing Surveys*, 51(1):4:1–4:27, 2018. doi: 10.1145/3143561. URL <https://doi.org/10.1145/3143561>.
- Martin D. Davis and Elaine J. Weyuker. Pseudo-oracles for non-testable programs. In *Proceedings of the ACM ’81 Conference*, pp. 254–257, 1981. doi: 10.1145/800175.809889. URL <https://doi.org/10.1145/800175.809889>.
- Saikat Dutta, Owolabi Legunsen, Zixin Huang, and Sasa Misailovic. Testing probabilistic programming systems. In *Proceedings of the 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, pp. 574–586, 2018. doi: 10.1145/3236024.3236057. URL <https://doi.org/10.1145/3236024.3236057>.
- Bradley Efron and Ronald Thisted. Estimating the number of unseen species: How many words did Shakespeare know? *Biometrika*, 63(3):435–447, 1976. doi: 10.1093/biomet/63.3.435. URL <https://doi.org/10.1093/biomet/63.3.435>.
- Stephen G. Eick, Catherine R. Loader, M. David Long, Lawrence G. Votta, and Scott Vander Wiel. Estimating software fault content before coding. In *Proceedings of the 14th International Conference on Software Engineering (ICSE)*, pp. 59–65, 1992. doi: 10.1145/143062.143090. URL <https://doi.org/10.1145/143062.143090>.

- William A. Gale and Geoffrey Sampson. Good–Turing frequency estimation without tears. *Journal of Quantitative Linguistics*, 2(3):217–237, 1995. doi: 10.1080/09296179508590051. URL <https://doi.org/10.1080/09296179508590051>.
- Marcus Gerhold and Mariëlle Stoelinga. Model-based testing of probabilistic systems. *Formal Aspects of Computing*, 30:77–106, 2018. doi: 10.1007/s00165-017-0440-4. URL <https://doi.org/10.1007/s00165-017-0440-4>.
- I. J. Good. The population frequencies of species and the estimation of population parameters. *Biometrika*, 40(3–4):237–264, 1953. doi: 10.1093/biomet/40.3-4.237. URL <https://doi.org/10.1093/biomet/40.3-4.237>.
- Ralph Guderlei and Johannes Mayer. Statistical metamorphic testing: Testing programs with random output by means of statistical hypothesis tests and metamorphic testing. In *Seventh International Conference on Quality Software (QSIC)*, pp. 404–409, 2007. doi: 10.1109/QSIC.2007.4385527. URL <https://doi.org/10.1109/QSIC.2007.4385527>.
- Douglas R. Hofstadter and Fluid Analogies Research Group. *Fluid Concepts and Creative Analogies: Computer Models of the Fundamental Mechanisms of Thought*. Basic Books, 1995. ISBN 9780465051540.
- Axel Legay, Benoît Delahaye, and Saddek Bensalem. Statistical model checking: An overview. In *Runtime Verification (RV)*, volume 6418 of *Lecture Notes in Computer Science*, pp. 122–135. Springer, 2010. doi: 10.1007/978-3-642-16612-9_11. URL https://doi.org/10.1007/978-3-642-16612-9_11.
- Qingzhou Luo, Farah Hariri, Lamyaa Eloussi, and Darko Marinov. An empirical analysis of flaky tests. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*, pp. 643–653, 2014. doi: 10.1145/2635868.2635920. URL <https://doi.org/10.1145/2635868.2635920>.
- James B. Marshall. *Metacat: A Self-Watching Cognitive Architecture for Analogy-Making and High-Level Perception*. PhD thesis, Indiana University, Bloomington, 1999. URL <https://science.slc.edu/jmarshall/metacat/dissertation.pdf>.
- James B. Marshall and Douglas R. Hofstadter. The Metacat project: A self-watching model of analogy-making. *Cognitive Studies: Bulletin of the Japanese Cognitive Science Society*, 4(4):57–71, 1997. doi: 10.11225/jcss.4.4_57. URL https://doi.org/10.11225/jcss.4.4_57.
- David A. McAllester and Robert E. Schapire. On the convergence rate of Good–Turing estimators. In *Proceedings of the 13th Annual Conference on Computational Learning Theory (COLT)*, pp. 1–6, 2000. URL <https://www.learningtheory.org/colt2000/papers/McAllesterSchapire.pdf>.
- William M. McKeeman. Differential testing for software. *Digital Technical Journal*, 10(1):100–107, 1998. URL https://www.bitsavers.org/pdf/dec/dtj/dtj_v10-01_1998.pdf.
- Melanie Mitchell. *Analogy-Making as Perception: A Computer Model*. MIT Press, 1993. URL <https://mitpress.mit.edu/9780262132893/analogy-making-as-perception/>.
- Ben O’Neill. Smallest covering regions and highest density regions for discrete distributions. *Computational Statistics*, 37:1229–1254, 2022. doi: 10.1007/s00180-021-01172-6. URL <https://doi.org/10.1007/s00180-021-01172-6>.
- Alon Orlitsky, Ananda Theertha Suresh, and Yihong Wu. Optimal prediction of the number of unseen species. *Proceedings of the National Academy of Sciences*, 113(47):13283–13288, 2016. doi: 10.1073/pnas.1607774113. URL <https://doi.org/10.1073/pnas.1607774113>.
- Ashwin Pananjady, Vidya Muthukumar, and Andrew Thangaraj. Just wing it: Near-optimal estimation of missing mass in a Markovian sequence. *Journal of Machine Learning Research*, 25(312):1–43, 2024. URL <https://jmlr.org/papers/v25/24-0511.html>.

- Krishna Patel and Robert M. Hierons. A mapping study on testing non-testable systems. *Software Quality Journal*, 26(4):1373–1413, 2018. doi: 10.1007/s11219-017-9392-4. URL <https://doi.org/10.1007/s11219-017-9392-4>.
- Sergio Segura, Gordon Fraser, Ana B. Sanchez, and Antonio Ruiz-Cortés. A survey on metamorphic testing. *IEEE Transactions on Software Engineering*, 42(9):805–824, 2016. URL <https://eprints.whiterose.ac.uk/id/eprint/110335/>.
- Elaine J. Weyuker. On testing non-testable programs. *The Computer Journal*, 25(4):465–470, 1982. doi: 10.1093/comjnl/25.4.465. URL <https://doi.org/10.1093/comjnl/25.4.465>.
- Uri Wilensky and William Rand. Making models match: Replicating an agent-based model. *Journal of Artificial Societies and Social Simulation*, 10(4), 2007. URL <https://www.jasss.org/10/4/2.html>. Article 2.
- Shin Yoo. Metamorphic testing of stochastic optimisation. In *Third International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*, pp. 192–201, 2010. doi: 10.1109/ICSTW.2010.26. URL <https://doi.org/10.1109/ICSTW.2010.26>.
- Håkan L. S. Younes and Reid G. Simmons. Probabilistic verification of discrete event systems using acceptance sampling. In *Computer Aided Verification (CAV)*, volume 2404 of *Lecture Notes in Computer Science*, pp. 223–235. Springer, 2002. doi: 10.1007/3-540-45657-0_17. URL https://doi.org/10.1007/3-540-45657-0_17.

A Reproduction and Evidence Boundaries

The analysis inputs are aggregate single-run reference counts and three archived check files containing single-run summaries and ordered port episodes. `tools/audit_numbers.py` reconstructs the p50 head using count order and lexicographic ties, validates stored flag lists against the counts, recomputes all probability baselines and analytic examples, and writes `number-audit.json`. It uses Python’s standard library and no external reference checkout. The reference record’s SHA-256 is recorded with the output, as are the check-file hashes. No synthetic observation is inserted into an engine measurement file.

`tools/audit_episodes.py` reconstructs last-success endpoints, no-answer and cap counts from the saved eight-run sequences, checks their consistency with stored flags, and writes `episode-audit.json`. It does not independently regenerate novelty labels from the unavailable historical episodic reference. The two audits have 14 standard-library unit tests in total.

```
python3 tools/audit_numbers.py
python3 tools/audit_episodes.py
python3 -m unittest discover -s tests -p 'test_*.py'
```

The LaTeX source bundle builds the manuscript, and an analysis supplement contains the measurement inputs, both arithmetic scripts, tests, and audit outputs. These support reproduction of the reported arithmetic. They are not an anonymous packaging of both full experimental implementations and are not a substitute for identifying the exact historically sampled reference build. The public source reconstruction bundle is also distinct from an anonymous review artifact. Author-identifying repository links are omitted here.

B Notation

Table 4: Notation distinguishes observed sets, unknown probabilities, and estimates.

Symbol	Meaning
p_R, p_P	Reference and port outcome laws for a fixed protocol
N_x, n_x	Reference and check sample sizes in runs or whole episodes, as specified
K, C_R	Number of check cycles and one-time reference cost
$C_{P,k}, C_{F,k}$	Check and follow-up costs for cycle k
L, ϕ	Episode horizon and fixed episode-outcome projection
J	Number of benchmark inputs, 19 in the case study
$c_x(o), f_{1,x}$	Reference outcome count and number of singleton outcomes
S_x, H_x, T_x	Observed reference set, selected empirical head, observed check set
$M_{R,x}, \widehat{M}_{R,x}$	Reference missing mass and its singleton-ratio estimate
$q_{P,x}$	Port probability of an outcome outside the frozen S_x
D_x, U_x	Number of novel draws and number of distinct novel outcomes

C Benchmark Inputs

Table 5 lists the complete benchmark. Up to three example answers per input are shared between the archived reference and post-repair CPU check, in decreasing reference-count order with lexicographic ties. They are observed outputs, not ground-truth labels.

Table 5: Benchmark inputs and observed shared answers; examples are not correctness labels.

Input	Initial	Modified	Target	Shared examples
misc4	a	b	z	b, z, y
fig5.7	aabc	aabd	ijkk	ijll, ijkl, ijl
misc3	abc	aabbcc	kkjjii	kkjjii, kkkjjjiii, kji
misc5	abc	abd	glz	gld, hlz, glz
misc2	abc	abd	ijk	ijl, ijd, ijk
run1	abc	abd	mrrjjj	mrrkkk, mrrjjk, mrrjjj
run4	abc	abd	xyz	xyd, wyz, xyz
misc1	abc	cba	mrrjjj	jjjrrm, mrrjjj, jrrmmm
fig5.4-top	eeqee	qeeq	xxixx	qeeq, ixxi, ixqx
eqe-baaab	eqe	qeq	abbba	baaab, qeeeq, qbbbq
run6	eqe	qeq	abbbc	qeeeq, baaaq, qcccb
run3	rst	rsu	xyz	xyu, wyz, xyz
run2	xqc	xqd	mrrjjj	mrrkkk, mrrjjk, mrrjjj
copy1	ab	c	ab	c, ab
copy2	bc	d	bc	d
copy3	xy	z	xy	z
copy4	zy	x	zy	x, zy
copy5	aabb	cc	aabb	cc, cac
copy6	abc	d	abc	d, abc